MODULE 19 · DOMAIN 7 - MAINTAIN INFRASTRUCTURE

Importing Existing Infrastructure

Bringing objects Terraform did not create under management, and the half of the job import does not do.

DOMAIN 7 of 8 COURSE WEIGHT 8% MODULE 19 of 27

admin@sthithapragna.com

Import creates a **binding in state**. It does not write your configuration. That half is still your job.

Why import exists

- Infrastructure that **predates Terraform**, or was created by hand during an incident.
- Objects created by **another tool** that you now want managed in one place.
- Recovering after somebody **removed a resource from state** and still needs it managed.
- Module four's last resort: rebuilding after **state was lost entirely**.
- **Not** for copying a resource between configurations. That is a state move, or a removed and an import.

> ON THE EXAM
>
> Import is adoption. If Terraform created the object and still tracks it, import is not the tool you want.

The terraform import command

- **terraform import ADDRESS ID**. The address is where it should live; the ID is the provider's identifier.
- It **writes state immediately**, with no plan and no preview.
- **One resource per invocation.** Forty resources means forty commands.
- **It does not generate configuration.** The resource block must already exist, or the next plan destroys the object.
- **It is not deprecated.** It suits one-off adoptions and quick repairs.

> ON THE EXAM
>
> This is the older route and it still works. Its weaknesses are the same as every state command: immediate, unreviewed, one at a time.

The import block

- **import** with a **to** address and an **id**. Available from Terraform 1.5.
- **It appears in the plan**, so you preview the import before state is modified.
- HashiCorp's wording: configuration-driven import is **predictable, works with CI/CD, and lets you preview**.
- **Many at once**, and with **for_each** from Terraform 1.7, a whole map in one apply.
- Once applied, the block is a no-op and can be removed, like a moved block.

> ON THE EXAM
>
> The plan reports the count as 'to import', separately from adds, changes and destroys. Recognise that line.

Command or block?

TERRAFORM IMPORT

Immediate and manual

- Writes state at once, no preview
- One resource per run
- Not version controlled
- Fine for a one-off or a repair

IMPORT BLOCK

Declarative and reviewed

- Appears in the plan first
- Many resources, and for_each
- Lives in configuration, goes through review
- Repeatable across environments

THE TIEBREAKER

Neither is deprecated, and both produce identical state entries. The difference is process, not result.

The import workflow

01

Identify

Find the object, and the provider-specific ID format from its docs.

02

Configure

Write a resource block, or generate a draft one.

03

Import

Add an import block and apply, or run terraform import.

04

Reconcile

Plan, and adjust configuration until it proposes nothing.

ON THE EXAM

The last step is where the real work is. An import is only finished when the plan is empty.

Generating configuration

- Write only the import block, then run **terraform plan -generate-config-out=FILE**.
- Terraform writes HCL containing its **best guess at the appropriate value** for each argument.
- **The file must be new.** Terraform will not overwrite an existing one.
- If your configuration has **no other resources for that provider**, add a provider block first, then run init again.
- **Experimental since 1.5.** HashiCorp says formatting and behaviour may change in later minor versions.

> ON THE EXAM
>
> Still marked experimental, and still the fastest way to start. Expect the output to change between versions.

What generated configuration looks like

- **Verbose.** Provider defaults, empty maps and explicit nulls that nobody would commit by hand.
- **Hardcoded.** Values that should be variables or references to other resources are literals.
- **Treat it as a draft.** Trim it, format it, re-plan, and only then commit.
- It is still far faster than reading a provider's documentation attribute by attribute.
- **All generated code needs review before committing.** That is the consistent advice everywhere.

> ON THE EXAM
>
> The output is a starting point, not a final artefact. Committing it unread is how a repository fills with provider defaults.

Where generation will not help

- **-generate-config-out cannot be used with an import block that uses for_each.**
- **Nor with resources that use count or for_each.** Terraform reports this as an error, not a warning.
- So the bulk import feature and the generation feature **do not combine**.
- For those, write the resource block yourself, then import with for_each.
- A workable route: generate for **one** instance, generalise it by hand, then import the rest.

> ON THE EXAM
>
> Two features that sound complementary and are mutually exclusive. This is a clean, checkable fact.

One object, one address

- **Each remote object should be imported to exactly one resource address.** Module four's rule.
- Import the same object twice and the mapping is ambiguous. Terraform may then act unexpectedly against it.
- The likely outcome: **two configurations both believe they own it**, and one destroys what the other manages.
- **Some services are represented by several resource types**, so full management may need several imports.
- Those are different objects, not the same object twice. Check the provider documentation.

> ON THE EXAM
>
> Creation guarantees the one-to-one rule automatically. Import is the one operation where you can break it.

Verifying an import

- **Run a plan. It should propose nothing.** That is the only real confirmation.
- **Changes mean your configuration does not match reality**, and applying would change the object.
- **A proposed destroy means the resource block is missing**, so Terraform sees a binding with nothing declared.
- **A proposed create means the address is wrong**, so the import landed somewhere your configuration is not looking.
- Iterate on the configuration, not on the state. State is already correct.

> ON THE EXAM
>
> Three plan outcomes, three different mistakes. Read which one you got before changing anything.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
After importing, the plan wants to destroy it	No resource block was written
After importing, the plan wants to change it	The configuration does not match reality yet
Adopt forty resources through a pipeline	import blocks, reviewed. Not forty commands
Generate the configuration for these	plan -generate-config-out, to a new file
Generation failed on a for_each import	Documented limitation. They do not combine
Two configurations manage the same object	It was imported twice. Break one binding

ON THE EXAM

Read what the post-import plan proposes. Destroy, change and create each point at a different mistake.

> An engineer adopts a manually created storage bucket. She looks up its identifier and runs terraform import for the address she intends to use. The command reports a successful import.
>
> She runs terraform plan, expecting no changes, and the plan proposes to destroy the bucket.

What happened?

A The import failed silently and left a partial state entry

B No resource block exists at that address, so Terraform sees a binding for something not declared

C The bucket must be tagged before Terraform will recognise it as managed

D terraform import always requires a second run to complete the adoption

◆ PAUSE AND DECIDE

What happened?

A Silent failure

The command reported success, and it succeeded. State now holds a correct binding, which is exactly why the plan can see something to destroy.

B No resource block

Correct. Import writes state and never writes configuration. A binding with nothing declared is, to Terraform, a resource you deleted.

C Tagging

Module four: tags were tried as an identity mechanism and abandoned. Terraform identifies objects by what state records.

D Two runs

Invents a rule. One invocation completes the import. What is missing is the other half of the job, which is yours.

THE SIGNAL

Module six's plan annotation says it exactly: destroyed 'because it is not in configuration'. That is what this plan is reporting.

> A team must adopt about forty existing resources across three accounts. The work must go through code review, run in their pipeline, and be repeatable in a second region later.
>
> One engineer proposes a runbook of forty terraform import commands. Another proposes import blocks.

Which fits the requirements, and why?

A The commands, since they take effect immediately and need no apply

B import blocks, because they are previewed in a plan, reviewed, and repeatable

C The commands, because import blocks cannot be used in automation

D Either. The two are functionally identical, so the choice is a matter of taste

◆ PAUSE AND DECIDE

Which fits the requirements, and why?

A Commands, immediate

Immediacy is the drawback. Forty unreviewed state mutations from a terminal is the opposite of what the requirements describe.

B import blocks

Correct. HashiCorp describes configuration-driven import as predictable, working with CI/CD, and previewable - which is the requirement list restated.

C Blocks cannot automate

Backwards. Working with CI/CD pipelines is one of the stated advantages of the block over the command.

D Functionally identical

The resulting state is identical; the process is not. The requirements here are entirely about process.

THE SIGNAL

The requirements name the block's three documented advantages almost word for word: reviewed, pipeline-friendly, repeatable.

> An engineer writes an import block using for_each over a map of twelve existing databases, intending to adopt them all in one apply. She then runs terraform plan with -generate-config-out to produce the resource configuration.
>
> Terraform reports an error before generating anything.

What is the cause, and what should she do?

A Generation cannot be used with for_each. Generate for one, generalise it, then import the rest

B The output file already exists and must be deleted first

C for_each on import blocks requires Terraform 1.9 or later

D Generation requires the resources to already exist in state

◆ PAUSE AND DECIDE

What is the cause, and what should she do?

A They do not combine

Correct. Terraform states the flag cannot be used with import blocks that use for_each, nor with resources using for_each or count.

B File exists

A real requirement - the file must be new - and it produces a different, clearly worded error about the file rather than about for_each.

C Needs 1.9

for_each on import blocks arrived in 1.7. Version is not the constraint here, and the exam tests 1.12 regardless.

D Must be in state already

Backwards. Generation exists precisely for resources not yet in state; once imported, no generation is needed.

THE SIGNAL

Bulk import and configuration generation are the two headline import features, and they are mutually exclusive.

> Two teams adopt Terraform independently. Both need a shared DNS zone, and each imports it into their own configuration so they can manage records.
>
> Months later, one team decommissions its configuration and runs destroy. The zone disappears, and the other team's next plan proposes to create it.

What was the underlying error?

A The second team should have run terraform refresh to detect the deletion earlier

B One remote object was imported into two addresses, so both configurations believed they owned it

C Shared resources must always use prevent_destroy on both sides

D The zone should have been imported into a CLI workspace shared by both teams

◆ PAUSE AND DECIDE

What was the underlying error?

A Refresh earlier

Detects the deletion sooner and prevents nothing. The zone is already gone by the time any refresh could notice.

B Imported twice

Correct. Each remote object should be imported to exactly one address. Two owners means one of them will eventually destroy it.

C prevent_destroy on both

Would have blocked this particular destroy, and M11 showed it dies when the resource block is deleted. It treats a symptom of dual ownership.

D A shared workspace

M18: a CLI workspace is a state file inside one working directory. Two teams with separate configurations cannot share one.

THE SIGNAL

The correct pattern is one owner and a data source for everyone else. Ownership, as module seven framed it, is the whole question.

Module 19 recap

- **Import creates a state binding. It never writes configuration.**
- **A post-import destroy plan means no resource block exists** at that address.
- **import blocks (1.5) are previewed, reviewed and repeatable.** The command is immediate and unreviewed.
- **-generate-config-out writes a draft**, to a new file, and is still experimental.
- **Generation does not work with for_each**, on the import block or on the resource.
- **One object, one address.** Import is the only operation that can break that rule.

> ON THE EXAM
>
> An import is finished when the plan is empty. Until then, fix the configuration, never the state.